

arc112F题解

题意：有 n ($n \leq 16$)种类型的卡片，以及 m ($m \leq 50$)种卡包，每个卡包有无限个。一开始有第 i 种卡片 c_i 个。可以进行如下两种操作任意次：一种是将一个卡包里的卡全部加进来，或者将 $2 * j$ 个 j 类型的卡换成一个 $(j + 1) \% n$ 的卡。问最后最少可能有多少卡。

先考虑一个贪心策略，假设第 j 个类型的卡不少于 $2 * j$ 张，那么把他们都换成 $j + 1$ 答案一定会变少。也就是说，我们可以把 $(c_1, c_2, \dots, c_n)$ 这个表示目前有多少卡牌的 n 元组，看成是一个有 n 位的 x 进制数，其中第 i 位的上限是 $2 * i$ ，可以理解成每达到 $2 * i$ 后就会往下一位进位1。

那么就可以定义如下哈希函数： $f(c_1, c_2 \dots c_n) = \sum_{j=1}^n 2^{j-1} (j-1)! c_j$ 。来实现能转换的状态的哈希值都相等。还有一个小细节是，如果最后一位满了，那么就会加到第一位上，也就是说，这个数值每达到 $2^n n!$ 就会变成1，可以看成是对 $2^n n! - 1$ 取模。因此，记 $N = 2^n n! - 1$ 。

发现一个状态对应了一个唯一的值，一个唯一的值也对应了一个状态，除了一个例外，就是 $(0, 0, \dots, 0)$ 和 $(1, 3, \dots, 2n - 1)$ 的哈希值都是0。但是题目本身不会出现前者，所以可以当成是一一对应的。还原方式也很简单，从高位到低位像拆分进制做即可。

裴蜀定理： a, b 是整数，令 $g = \gcd(a, b)$ ，对于任意整数 x, y 都有 $ax + by$ 是 g 的倍数，并且一定存在一组 x, y 使得 $ax + by = g$ 。前半句话显然。对于后半部分，使用拓展欧几里得即可找到一组解。将该定理拓展到 n 个数时，可以进行 $n - 1$ 次两两合并，然后最后的 g 就是所有数的gcd。

然后考虑卡包的影响，我们对每个卡包也算出他们的哈希值。加入一个卡包相当于给哈希值加上对应的值。由于是在模 N 一下，卡包数量也是无限的，那么能产生的哈希值就是 $(N, s_1, s_2, \dots, s_n)$ 这些元素的线性组合。由裴蜀定理得，令 $g = \gcd(N, s_1, s_2, \dots, s_n)$ ，在模 N 意义下能产生所有 g 的倍数的值。而我们要求的，就是在能产生的所有模 g 意义下跟初始状态相同的值中，找到每个状态对应的 $c_1 + c_2 + \dots + c_n$ 最小的那个值。

然后最后可能的 g 一定是 N 的一个因数。然后令 $B = 2 * 10^6$ ，打表发现，所有可能的 N 的所有因数，要么都是 $\leq B$ 的，否则就是 $\geq \frac{N}{B}$ 的。

对于后者，可以暴力枚举算贡献，由于最多跳 B 次，所以复杂度是 $O(BN)$ 的。

对于前者，有两种处理方法。第一种是跑一个背包， $dp(i, j)$ 表示用前 i 位，凑出模 g 最小需要多少个。转移枚举当前放几个并更新，复杂度 $O(BN^2)$ 。

另一种是通过最短路思想， $dp(i)$ 表示模 g 意义下凑出 i 最小需要多少个。由于边权都是1，可以通过bfs更新。更新时枚举下一个放什么来实现。复杂度 $O(BN)$ 。

Code(chb1975)

```
#include <bits/stdc++.h>
#define rep(i,n) for ((i)=1;(i)<=(n);(i)++)
using namespace std;
int n,m,i,j;
long long coef[24];
int f[1214835],f1[1214835];
int calc(long long x){
    int i,s=0;
    for(i=n;i>=1;i--){
        s+=x/coef[i];
        x-=x/coef[i]*coef[i];
    }
    return s;
}
```

```

int main(){
    cin>>n>>m;
    long long md=1;
    rep(j,n){
        coef[j]=md;
        md*=2*j;
    }
    md--;
    long long s=0;
    rep(i,n){
        int x;
        cin>>x;
        s+=coef[i]*x;
    }
    int ans=calc(s);
    s%=md;
    long long g=0;
    while(m--){
        long long t=0;
        rep(i,n){
            int x;
            cin>>x;
            t+=coef[i]*x;
        }
        t%=md;
        t=__gcd(t,md);
        g=__gcd(g,t);
    }
    if(g==0||g==md){
        cout<<ans<<endl;
        return 0;
    }
    if(g>1214827){
        s%=g;
        while(s<=md){
            int t=calc(s);
            if(t) ans=min(ans,t);
            s+=g;
        }
        cout<<ans<<endl;
        return 0;
    }
    memset(f,0x3f,sizeof(f));
    rep(i,n){
        memcpy(f1,f,sizeof(f1));
        for(j=1;j<i+i;j++){
            f1[j*coef[i]%g]=min(f1[j*coef[i]%g],j);
        }
        for(j=0;j<g;j++){
            int k;
            rep(k,i+i-1){
                f1[(j+k*coef[i])%g]=min(f1[(j+k*coef[i])%g],f[j]+k);
            }
        }
        memcpy(f,f1,sizeof(f));
    }
    cout<<min(ans,f[s%g])<<endl;
    return 0;
}

```

```
}
```

Code(nuip)

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    long n, m;
    cin >> n >> m;
    const auto read = [&]() {
        long sum = 0, prod = 1;
        for (int i = 0; i < n; ++i) {
            long x;
            cin >> x;
            sum += x * prod;
            prod *= 2 * (i + 1);
        }
        return sum;
    };
    const long init = read();
    vector<long> ss(m);
    for (auto &s : ss)
        s = read();

    long N = 1;
    for (int i = 1; i <= n; ++i)
        N *= 2 * i;
    --N;
    const long g = accumulate(ss.begin(), ss.end(), N, gcd<long, long>);

    if (g < N / g) {
        vector<long> edges(n + 1);
        edges[0] = 1;
        for (int i = 0; i < n; ++i)
            edges[i + 1] = edges[i] * 2 * (i + 1);
        vector<int> dist(g);
        queue<int> que;
        for (const auto &e : edges) {
            dist[e % g] = 1;
            que.emplace(e % g);
        }
        while (que.size()) {
            const auto v = que.front();
            que.pop();
            for (const auto e : edges) {
                if (const auto u = (v + e) % g; dist[u] == 0) {
                    dist[u] = dist[v] + 1;
                    que.emplace(u);
                }
            }
        }
        cout << dist[init % g] << endl;
    } else {
        auto ans = numeric_limits<long>::max();
        for (long i = init % g; i <= N; i += g) {
```

```
    if (!i)
        continue;
    long rem = i;
    long sum = 0;
    for (int j = 1; j <= n; ++j) {
        const auto t = 2 * j;
        sum += rem % t;
        rem /= t;
    }
    ans = min(ans, sum);
}
cout << ans << endl;
}
return 0;
}
```